

Федеральное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: «Вычислительная математика»

Лабораторная работа №1

«Решение системы линейных алгебраических уравнений СЛАУ»

Вариант 10 – метод Гаусса-Зейделя

Выполнила:

Студентка группы Р3213 Пименова Екатерина Андреевна

Преподаватель:

Машина Екатерина Алексеевна

г. Санкт-Петербург
2025

Оглавление

<u>ЦЕЛЬ РАБОТЫ</u>	<u>3</u>
<u>ОПИСАНИЕ МЕТОДА</u>	<u>4</u>
<u>ЛИСТИНГ ПРОГРАММЫ НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ PYTHON</u>	<u>5</u>
<u>ОПИСАНИЕ ИСХОДНЫХ ДАННЫХ:</u>	<u>6</u>
<u>РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ:</u>	<u>7</u>
ПРИМЕР 1	7
ПРИМЕР 2	8
<u>ВЫВОД</u>	<u>9</u>

Цель работы

Цель работы – изучение вычислительного метода решения системы алгебраических линейных уравнений СЛАУ, а также реализация его на ЭВМ на одном из выбранных языков программирования.

Описание метода

Метод Гаусса-Зейделя является итерационным методом решения СЛАУ. Он модифицирует метод простых итераций за счет использования на $k+1$ итерации алгоритма уже полученных на этом же шаге значений.

Расчетные формулы:

$$\begin{aligned} x_1^{(k+1)} &= c_{11}x_1^{(k)} + c_{12}x_2^{(k)} + \dots + c_{1n}x_n^{(k)} + d_1 \\ x_2^{(k+1)} &= c_{21}x_1^{(k+1)} + c_{22}x_2^{(k)} + \dots + c_{2n}x_n^{(k)} + d_2 \\ x_3^{(k+1)} &= c_{31}x_1^{(k+1)} + c_{32}x_2^{(k+1)} + c_{33}x_3^{(k)} \dots + c_{3n}x_n^{(k)} + d_3 \\ &\dots\dots\dots \\ x_n^{(k+1)} &= c_{n1}x_1^{(k+1)} + c_{n2}x_2^{(k+1)} + \dots + c_{nn-1}x_{n-1}^{(k+1)} + c_{nn}x_n^{(k)} + d_n \end{aligned}$$

Рабочая формула метода Гаусса-Зейделя:

$$x_i^{(k+1)} = \frac{b_i}{a_{ii}} - \sum_{j=1}^{i-1} \frac{a_{ij}}{a_{ii}} x_j^{k+1} - \sum_{j=i+1}^n \frac{a_{ij}}{a_{ii}} x_j^k \quad i = 1, 2, \dots, n$$

Итерационный процесс продолжается до тех пор, пока:

$$|x_1^{(k)} - x_1^{(k-1)}| \leq \varepsilon, \quad |x_2^{(k)} - x_2^{(k-1)}| \leq \varepsilon, \quad |x_3^{(k)} - x_3^{(k-1)}| \leq \varepsilon$$

Листинг программы на языке программирования Python

```
import os

global N, EPS

def print_M(M):
    for i in M:
        for j in i:
            print(str(round(j, 4)).center(6), end=" ")
        print()
    print()

def read_from_file():
    with open("matrix.txt") as f:
        n = int(f.readline())
        if n > 20:
            raise ValueError

        eps = float(f.readline())
        M = [i.rstrip().split(" ") for i in f.readlines()]
    return eps, n, M

def validate_matrix(M):
    if len(M) != N:
        raise ValueError

    for i in range(len(M)):
        if (len(M[i])) != N + 1:
            raise ValueError
        M[i] = list(map(float, M[i]))
    return M

# в матрице N строк и N+1 столцов
def check_diagonal_dominance(M):
    for i in range(N):
        if not (abs(M[i][i]) >= sum([abs(M[i][j]) for j in range(N) if j
!= i])):
            return False
    return True

def change_to_get_diagonal_dominance(M):
```

```

X = ['x' + str(i) for i in range(N)] # чтоб сохранить коэфф при
переменных
for i in range(N):
    for j in range(i, N):
        M[i], M[j] = M[j], M[i]

    # поменяла местами строки и теперь для каждой такой новой
    матрицы надо поменять местами столбцы

    for k in range(N):
        for h in range(k, N):
            for _ in range(N):
                M[_][k], M[_][h] = M[_][h], M[_][k]

            if check_diagonal_dominance(M):
                X[k], X[h] = X[h], X[k]
                return X, M

return None, None

def count_norm(M):
    res = 0
    for i in M:
        res = max(res, sum([abs(j) for j in i]))

    return res

def get_C_D_matrix(M):
    C = []
    D = []
    for i in range(N):
        a_ii = M[i][i] if M[i][i] != 0 else 1
        c = []
        for j in range(N + 1):
            if j == N:
                D.append(M[i][j] / a_ii)
                continue
            if i == j:
                c.append(0)
            else:
                c.append(-1 * M[i][j] / a_ii)
        C.append(c)

    return C, D

```

```

def read_M():
    start = input("введите f, если хотите прочитать данные из файла
matrix.txt и любой другой символ в ином случае:")

    if start.strip() == "f" and os.path.exists("matrix.txt"):
        try:
            eps, n, M = read_from_file()
            return eps, n, M

        except ValueError:
            print("невалидные входные данные...\nзавершение работы
программы..")
            exit(1)
    else:
        n = int(input("введите размерность матрицы (≤20): "))
        eps = float(input("введите точность: "))
        print("введите матрицу:")
        M = [input().split(" ") for _ in range(n)]
        return eps, n, M

def get_tochnost(x_k, x_k1):
    # max(abs(x_k_i - x_k1_i))
    a = [abs(x_k[i] - x_k1[i]) for i in range(len(x_k))]
    return max(a)

def put_in_correct_order(x_k, X):
    res = [0] * N
    for i in range(N):
        res[i] = round(x_k[X.index(f'x{i}')] , 5)
    return res

def gauss_zeidel(C, D, X):
    print("\nРАСЧЁТЫ ПО МЕТОДУ ГАУССА-ЗЕЙДЕЛЯ:\n")
    k = 0
    # x_k это D
    # x_{k+1} который мы счa будем заполнять это

    x_k = D # имеет вид d1 d2 d3
    x_k1 = [0] * N
    print(f"    номер итерации      | {'x^(k)'.center(32)} | max(|x^(k)_i -
x^(k+1)_i|) ")
    print("-" * 90)
    print(f"          0              | {str(put_in_correct_order(x_k,
X)).center(32)} | -              ")

```

```

while k == 0 or not (get_tochnost(x_k, x_k1) <= EPS):
    x_k, x_k1 = x_k1, [0] * N
    # print(f"x({k})", x_k)
    for i in range(N):
        x = 0
        # всего N шагов и на i-том мы берем i-1 слагаемых из x_k+1
        for j in range(i, N):
            x += C[i][j] * x_k[j]

        for h in range(0, i):
            x += C[i][h] * x_k1[h]

        x += D[i]
        x_k1[i] = x
    # здесь получили x_k1
    k += 1
    print(
        f"          {k}          | {str(put_in_correct_order(x_k1,
X)).center(32)} |          {round(get_tochnost(x_k, x_k1), 4)}          ")

    return k, x_k1

def main():
    global N, EPS
    try:
        EPS, N, M = read_M()
        M = validate_matrix(M)
        print("БЫЛО")
        print(''.join(['x' + str(i)).center(6) for i in range(N)])
        print_M(M)
        X, diagonalized_M = change_to_get_diagonal_dominance(M)

        if not diagonalized_M:
            print("невозможно добиться диагонального преобладания(..\n")
            print("проверяем значение нормы матрицы", count_norm(M))
            if count_norm(M) >= 1:
                print("ну вот( условие сходимости не выполнено(\nсистема
не сойдется к ответу данным численным методом ")
                "или будет делать это ОЧЕНЬ долго(")
                exit(1)
            print("ура! условие сходимости выполнено! продолжаем..")
            print()
        if diagonalized_M:
            print("СТАЛО для диагонального преобладания")
            print(''.join([x.center(6) for x in X]))
            print_M(diagonalized_M)

```



```

C, D = get_C_D_matrix(diagonalized_M)
print("получаем матрицу C")
print_M(C)
print("и столбик свободных коэфф D:\n" + '\n'.join(list(map(str,
D))))

k, x_k1 = gauss_zeidel(C, D, X)

print("ИТОГО:")
x_k1 = put_in_correct_order(x_k1, X)

print(
    f"\t-потребовалось {k} итераций \n\t-ответ с учетом
погрешности: {x_k1}\n\t-округленный ответ: {list([round(i, 3) for i in
x_k1])}")

except ValueError:
    print("невалидные входные данные...\nзавершение работы
программы..")
    exit(1)

main()

```

Описание исходных данных:

введите f, если хотите прочитать данные из файла matrix.txt и любой другой символ в ином случае:

введите размерность матрицы (≤ 20): 3

введите точность: 0.01

введите матрицу:

5 -2 3 10

-3 9 1 5

2 -1 7 7

Пользователь вводит размерность матрицы, точность вычислений и саму матрицу. Ввод доступен и из консоли, и из файла.

ПРИМЕР 2

gauss-zeidel.py matrix.txt x

1 3
2 0.01
3 2 2 10 14
4 10 1 1 12
5 2 10 1 13
6

↑
↓
↺
↻
🖨
🗑

введите f, если хотите прочитать данные из файла matrix.txt и любой другой символ в ином случае: *f*
БЫЛО
x0 x1 x2
2.0 2.0 10.0 14.0
10.0 1.0 1.0 12.0
2.0 10.0 1.0 13.0
СТАЛО для диагонального преобладания
x2 x1 x0
10.0 2.0 2.0 14.0
1.0 10.0 1.0 12.0
1.0 2.0 10.0 13.0
получаем матрицу C
0 -0.2 -0.2
-0.1 0 -0.1
-0.1 -0.2 0
и столбик свободных коэфф D:
1.4
1.2
1.3
РАСЧЁТЫ ПО МЕТОДУ ГАУССА-ЗЕЙДЕЛЯ:

номер итерации	x^(k)	max(x^(k)_i - x^(k+1)_i)
0	[1.3, 1.2, 1.4]	-
1	[0.948, 1.06, 1.4]	1.4
2	[0.99909, 1.00536, 0.9984]	0.4016
3	[1.00005, 1.00018, 0.99911]	0.0052

ИТОГО:
-потребовалось 3 итераций
-ответ с учетом погрешности: [1.00005, 1.00018, 0.99911]
-округленный ответ: [1.0, 1.0, 0.999]
Process finished with exit code 0
|

Вывод

Метод Гаусса-Зейделя приятный. Мне понравилось.